

# 家庭作业：人工神经网络与人脸照片

15-681: 机器学习

Tom Mitchell

卡内基梅隆大学

截止日期: 1997 年 10 月 16 日

## 1 简介

本作业让你有机会把针对神经网络的学习应用到人脸识别的问题上。它分为两个部分。第一部分你必须独立完成，使用神经网络程序训练出一个太阳镜识别器、一个人脸识别器和一个姿势识别器。第二部分是可选的，你可以和 1 到 2 个其他学生组队来研究你们自己选择的问题。你后面会用到的人脸照片来自于之前的机器学习课程中的学生的人脸。

要完成本作业，你并不需要做太多的代码编写工作，也别被本题的篇幅所吓倒，但是训练你的网络的确需要花费很多时间。非常推荐你先整体阅读完本题并尽早开始。

### 1.1 脸部图片

图片数据可以从 `/afs/cs/project/theo-8/faceimages/faces` 找到。这个目录下包含了 20 个子目录，每个目录对应一个人并使用用户标识进行命名。每个这样的目录下又包含了同一个人的几张不同的脸部图片。

值得你注意的是使用下面这种命名约定的图片：

`< 用户标识 >_< 姿势 >_< 表情 >_< 眼睛 >_< 比例 >.pgm`

- `< 用户标识 >` 是图片中人物的用户的一个编号，并且这个字段有 20 个可能的值，分别是：an2i、at33、boland、bpm、ch4f、cheyer、choon、danieln、glickman、karyadi、kawamura、kk49、megak、mitchell、night、phoebe、saavik、steffi、sz24 以及 tammo。
- `< 姿势 >` 是人物的头部位置，本字段有 4 个可能的值，分别是：straight、left、right、up。
- `< 表情 >` 是人物的面部表情，本字段有 4 个可能的值，分别是：neutral、happy、sad、angry。
- `< 眼睛 >` 是人物的眼部状态，本字段有 2 个可能的值，分别是：open、sunglasses。
- `< 比例 >` 是图片的比例，本字段有 3 个可能的值，分别是：1、2 和 4。1 代表全分辨率的图片（128 列 × 120 行）；2 表示半分辨率的图片（64 × 60）；4 表示四分之一分辨率的图片（32 × 30）。在本题中，你会使用四分之一分辨率的图片来做实验，从而让训练时间可控。

如果你仔细查看图片目录，你可能已经注意到了有一些图片的后缀名是 `.bad` 而不是 `.pgm`。原因是在 640 张图片中，有 16 张由于相机设置的问题而导致了一些瑕疵；这些就是后缀名为 `.bad` 的图片。有些人的瑕疵会比另一人更多，但每个只要被拍摄了“脸部”的人都会至少有 28 张好的面孔图片（不算比例的话，一共有 32 张不同的可能图片）。

## 1.2 查看脸部图片

要查看图片,你可以使用 `xv` 程序。在 Andrew 机器中它位于 `/usr/local/bin/xv`,而在 CS 机器中,它可能位于 `/usr/misc/.X11R6/bin/xv` 或者 `/usr/local/bin/xv`。`xv` 可以处理多种不同的图片格式,包括我们的脸部图片存储所使用的 PGM 格式。我们不会在本文中详细讨论 `xv`,但是我们会快速描述一下你需要知道的 `xv` 的基本使用方法。

要启动 `xv`,只需要从命令行指定一个或者多个图片,就像这样:

```
xv /afs/cs/project/theo-8/faceimages/faces/glickman/glickman_straight_happy_open_4.pgm
```

这会弹出一个 X 窗口来显示脸部。点击图片窗口的右边按钮会打开或者隐藏包含着多个按钮的控制面板。Dbl Size 按钮会在你每点击一次时就将图片的显示尺寸放大两倍。想必你已猜到这在查看四分之一分辨率的图片时非常有用。

你还能通过在图片窗口上移动指针时,按住左边的按钮来获取像素值。一个文本条会出现,并显示指针定位在图片上的坐标以及亮度值。

要退出 `xv`,只需要点击 Quit 按钮或者在某一个 `xv` 窗口中键入 `q` 即可。

## 1.3 神经网络和图片访问代码

我们提供一个 3 层的全连接前馈神经网络的 C 代码,使用反向传播算法来调整权值。并且我们还提供一个图片包来访问这些脸部图片,以及可以由你修改的训练和测试高级程序骨架代码,从而让实验更容易。为了帮助探索网络具体学习到了什么,你还可以找到一个实用工具程序来将隐藏单元的权值可视化成为一个图片。

代码位于 `/afs/cs/project/theo-8/faceimages/code`。拷贝这里的所有文件到你自己的目录中,然后输入 `make`。注意:要小心使用 `cp *` 而不是 `cp *.*` 以确保你也复制了 `Makefile`。当编译完成,你就有了一个可执行程序: `facettrain`。简单来说, `facettrain` 会将图片文件列表做为输入,并且使用它们做为神经网络的训练和测试集。 `facettrain` 可被用来训练以及/或者识别,还有将网络保存为文件的能力。

代码在 CS-side Alphas、DecStations、Sun SPARC-2s 和 IBM PowerPC's 以及 Andrew-side DecStations 和 Sun SPARC-5s 上被成功的编译和测试过了。如果你想在其他平台上使用这个代码,尽管去做,只是要了解代码仅仅在前述平台上测试过。

在本文的 3 一节可以找到详细的用法说明、源文件解释和相关的其他信息。

# 2 作业

## 2.1 第一部分 (必做)

对于下面的初始实验中的问题,按顺序为你的解答做一个小结。

1. 为本题在你的家目录中运行以下命令来获得训练和测试集数据:

```
cp /afs/cs/project/theo-8/faceimages/trainset/*.list .
```

2. 给到你的代码现在已经被设置成用来学习使用用户标识 `glickman` 来识别人物。修改代码以实现一个“sunglasses”识别器;即,训练出一个神经网络,当给定一个图片做为输入时,给出图片中的脸部是否戴着太阳镜。参考第 3 节的概览以了解如何修改这段代码。

3. 使用默认的学习参数(学习率 0.3,冲量 0.3)通过 75 个周期训练出一个网络,使用如下命令:

```
facettrain -n shades.net -t straightrnd_train.list -l straightrnd_test1.list  
-2 straightrnd_test2.list -e 75
```

`facettrain` 的参数描述详见第 3.1.1 节,这里只按顺序给出一个简短的介绍。`shades.net` 是当训练结束时保存的网络文件名。`straightrnd_train.list`、`straightrnd_test1.list` 和 `straightrnd_test2.list` 是用来分别指明训练集(70 个样例),以及两个测试集(一个是 34 个样例,另一个是 52 个样例)的文本文件。

这个命令从 156 个“直接”图片中随机选择的 70 个样本上创建和训练你的网络，然后分别从剩下的其他样本中随机挑选 34 和 52 个样本上做测试。这样测试策略可以这样来理解，即大约  $\frac{1}{3}$  的图片 (`straightrnd_test2.list`) 被保留用来做测试。其余的  $\frac{2}{3}$  被用来做训练以及交叉验证。而这其中又有  $\frac{2}{3}$  的图片被保留作为训练集 (`straightrnd_train.list`)，以及  $\frac{1}{3}$  被用作决定什么时候停止训练的验证集 (`straightrnd_test1.list`)。

- 你修改了什么代码？训练集所达到的最大分类精度是多少？达到这个水平用了多少个训练周期？验证集呢？测试集呢？注意如果你使用相同的参数和相同的输入将同样的系统再跑一遍的话，你会得到一模一样的结果，这是因为代码默认每次都使用了相同的随机数产生器种子。你需要仔细阅读第 3.1.2 节才能解释你的实验并回答这些问题。
- 现在，实现一个 20 到 1 的人脸识别器；比如，实现一个神经网络，它接受一个图片做为输入，而输出是人物的用户标识。要做到这样的效果，你会需要实现一个不同的输出编码（因为你现在必须能够从 20 个人物中做区分）。(提示：将学习速率和冲量都设置成 0.3，并且使用 20 个隐藏单元)。

- 像之前一样，训练这个网络，但这次使用 100 个训练周期：

```
facetrain -n face.net -t straighteven_train.list -1 straighteven_test1.list  
-2 straighteven_test2.list -e 100
```

你也许会想，为什么你只需要在一个受限范围（“直接”图片）的样本上进行训练。核心原因是训练时长。如果你有一个非常快的机器（如果比 Alpha 或者 Sun4 还慢，那就实在太慢了），那么非常欢迎你在整个样本集上进行实验（在上面的命令中将 `straight` 替换成 `all` 即可）。否则，只需要使用“直接”图片。

`straightrnd_*.list` 和 `straighteven_*.list` 集的区别在于前者纯粹是随机地将图片分为训练和测试集，而后者在区分时将确保集合中的每个人物都相对平均分布。因为对于每个人物我们仅仅只有 7 或者 8 个“直接”图片，所以如果不能够平均分配它们的话，就可能会造成使用那些被训练得最少的面孔来对网络进行测试。

- 这一次代码中的哪一部分需要被修改呢？你如何对输出进行编码？在训练集中能够达到的最高精度是多少？达到这个最高精度需要多少训练周期？验证和测试集呢？
- 现在让我们来更加仔细地审视哪些图片会让网络分类错误：

```
facetrain -n face.net -T -1 straighteven_test1.list -2 straighteven_test2.list
```

这些被错误归类的图片有什么特别的共性吗？

- 实现一个姿态识别器；比如，实现这样一个神经网络，当给定一个图片做为输入时，指示了图片中的人究竟是在向前看、向上看、向左看还是向右看。你也需要实现一个不同的编码来完成这个任务。(提示：将学习速率和冲量设置成 0.3，并使用 6 个隐藏单元)。
- 训练网络 100 个轮次，这次将样本设置为所有图片：

```
facetrain -n pose.net -t all_train.list -1 all_test1.list  
-2 all_test2.list -e 100
```

因为相比于人脸识别器来说，姿态识别器网络应该拥有比较少的权重来被更新，所以即使你用比较慢的机器也可以体验使用所有图片的快乐。在这一任务中，260 个样例都在训练集中，140 个样例在 `test1` 中而 193 个在 `test2` 中。

- 你这一次该如何编码输出？训练集能够达到的最高精度是多少？需要多少个训练周期达到这个水平？每个测试集呢？
- 现在，尝试审查一下反向传播针对每一个像素又是如何调整隐藏单元的权值的。首先在你的系统中敲入 `make hidtopgm` 来编译工具。然后，要将隐藏单元  $n$  的权值可视化出来的话，敲入：

```
hidtopgm pose.net image-filename 32 30 n
```

对图片 *image-filename* 调用 `xv` 应该能够显示出权值的范围，最小的权值对应像素值为 0 的像素，而最大的权值对应像素值为 255 的像素。如果图片看起来正好像是噪点，那么尝试使用 `facetrain_init0` 重新训练（用 `make facetrain_init0` 来编译），它会将隐藏单元的权值初始化成 0 到一个新的网络中，而不是随机值。

13. 隐藏单元看起来会在图片的某一区域拥有明显比别的地方更高的权值吗？有没有一些隐藏单元看起来像是针对某一特征做了一些微调？

## 2.2 第二部分（选做!）

现在你已经知道如何使用 `facetrain` 了，是时候整点儿活了。找一两个其他的学生成立一个团队，挑几个有意思的话题——要有创意！做一些实验，并且准备一个简短的稿子来展示你的团队的主意、实验结果、以及你得到的任何结论（几页就足够了）。对于这部分作业，你的团队出一份团队文稿就行。

一些可能的实验（如果你想尝试别的，那就别限制了自己）是：

- 将姿态识别器的输出作为人脸识别的输入，看看这会怎样影响最后的表现。要做到这点你需要一个机制来保存姿态识别器的输出单元以及一个加载这些数据到人脸识别器的机制。
- 学习图片中的某些特征的位置 信息，比如眼睛。你可以使用 `xv` 来得到每个图片中感兴趣的特征的坐标值，然后将它们做为目标值。
- 从下面的地址查看额外的以前班级的数据  
`/afs/cs.cmu.edu/project/theo-8/ml94faces.`  
你可以使用什么技术来训练网络以达到从一个数据集到另一个的更好的泛化效果？
- 网络在一次训练多个概念时表现得怎么样？隐藏层为多个概念形成的表征会相互干扰吗？又或者甚至互相修改？
- 使用图片包、权值可视化工具、以及/或者任何其他你有的东西来尝试更好地理解网络到底学习到了什么。利用这个信息，你觉得网络究竟在学习什么？你能利用这个信息来改进泛化吗？
- 修改输入或者输出的编码方式来尝试改进泛化精度。
- 采用不同数量的隐藏单元、训练样例数量、训练周期数、冲量以及学习速率，或者任何其他你想要尝试的，来实现最大可能的训练和测试集精度的区分度（比如，你能将网络过拟合到什么程度），以及最小可能的区分度（比如，你能达到的最好表现是什么）。

## 3 文档

本题的代码分为几个模块：

- `pgmimage.c`, `pgmimage.h`: 图片包。支持 PGM 图片文件的读/写，以及像素的读取/写入。提供 `IMAGE` 的数据结构，以及 `IMAGELIST` 数据结构（指向图片的指针数组；在处理多个图片时非常有用）。**你不需要修改这个模块中的任何代码就能完成本作业。**
- `backprop.c`, `backprop.h`: 神经网络包。支持三层全连接前馈网络，使用反向传播算法来调整权值。提供高层程序来创建、训练、以及使用网络。**你不需要修改这个模块中的任何代码就能完成本作业。**
- `imagenet.c`: 加载图片到网络的输入单元的接口程序，并且设置训练的目标向量。当要实现人脸识别器和姿态识别器时，你需要修改程序 `load_target`，来为你所选择的输出编码设置合适的目标向量。

- `facetrain.c`: 用来实现“TA”识别器的使用所有以上模块的顶层程序。你需要修改这段代码来改变网络大小以及学习参数，不过都是非常小的修改。`performance_on_imagelist()` 和 `evaluate_performance()` 是表现评估程序，也位于这个模块中；你需要为你的人脸识别器和姿态识别器修改它们。
- `hidtopgm.c`: 隐藏单元权重可视化工具。尽管探索一些不同的替代可视化方案非常有趣，但这里不需要做任何修改。

尽管你只需要修改 `imagenet.c` 和 `facetrain.c` 中的代码，但是只要你想，你可以修改任何文件中的任何代码，只要能够让你做实验时更加方便就行。

## 3.1 facetrain

### 3.1.1 运行 facetrain

`facetrain` 有几个选项可从命令行指定。这一节简单介绍每个选项都是如何工作的。关于这段信息的一个非常简短的总结可以通过运行不带任何参数的 `facetrain` 来得到。

- n <network file> - 这个选项要么加载一个已经存在的网络文件，要么使用给定的名字创建一个新的文件。训练结束后，神经网络会被存储在这个文件中。
- e <number of epochs> - 这个选项指定将要运行的训练轮次。如果没有被指定，就默认为 100。
- T - 为仅测试模式（没有训练）使用。会为指定的 3 个数据集分别报告性能，并且那些被错误分类的图片会随着相应的输出单元水平被列出来。
- s <seed> - 会被用作随机数产生器种子的一个整数。默认种子是 102194（猜猜我在写这篇文章时是什么日子）。通过生成相同序列的随机数，可以让你在必要时重现实验。这也允许你通过修改种子来尝试一组不同的随机数。
- S <number of epochs between saves> - 这个选项指定了存档之间的训练轮次数。默认为 100，这意味着如果你训练 100 个轮次（默认如此），网络只在训练完成时保存。
- t <training image list> - 该选项指定了一个文本文件，其包含一系列图片的路径名，一行一个，它们会被用作训练集。如果本选项没有指定，就假设训练不会发生（即相当于 `epochs = 0`），并且网络会直接在测试集上运行。在这样的情况下，训练集的统计指标会全部都是 0。
- 1 <test set 1 list> - 这个选项指定了一个文本文件，其包含一系列图片的路径名，一行一个，它们会被用作测试集。如果这个选项没有被指定，测试集的统计指标会全部都是 0。
- 2 <test set 2 list> - 和上面的一样，但是用作第二个测试集。之所以会有使用两个测试集的想法，是因为其中一个可以用作训练/测试范式的一部分，在这个范式下，当测试集上的表现出现下降时就停止训练。而另一个测试集则被用作最终网络的“真正”的测试。

### 3.1.2 解读 facetrain 的输出

当你运行 `facetrain` 时，它会先读取所有的数据文件并打印出一系列的行输出来报告这些操作。一旦所有数据都被装载好了，它就开始训练。在这个节骨眼儿上，每一个训练周期都会有一行总结输出，显示网络在训练集和测试集上的表现。对每一个训练周期，如下性能指标会被输出：

```
<epoch> <delta> <trainperf> <trainerr> <t1perf> <t1err> <t2perf> <t2err>
```

这些值有如下含义：

`epoch` 是刚完成的训练轮次的序数；如果这个序数是 0，就代表还没有开始执行训练。

`delta` 是针对该训练轮次中所有训练样例在反向传播过程中所计算的隐藏以及输出单元的所有  $\delta$  值的总和。

`trainperf` 是正确分类的样例在训练集上的百分比。

`trainerr` 是对所有的训练样例，使用误差函数  $\frac{1}{2} \sum (t_i - o_i)^2$  计算出来的平均值，其中  $t_i$  是输出单元  $i$  的目标值，而  $o_i$  是这个单元的实际输出值。

`t1perf` 是正确分类的样例在第一个测试集中的百分数。

`t1err` 是第一个测试集中所有的样例，使用上面描述的误差函数计算出来的平均值。

`t2perf` 是正确分类的样例在第二个测试集中的百分数。

`t2err` 是第二个测试集中所有的样例，使用上述误差函数计算出来的平均值。

## 3.2 提示

虽然你不需要修改图片或者网络包，但是你需要了解一点它们中的程序和数据结构，这样才能轻松地为你的网络实现新的输出编码。下面的小节中会描述每个包的一些细节。你可以查看 `imagenet.c`、`facetrain.c` 和 `facerec.c` 来了解程序实际上是如何使用的。

事实上，先看一下 `facetrain.c` 是一篇好文章，从而了解到了训练过程是怎么工作的。你会注意到 `imagenet.c` 中的 `load_target()` 被调用是去为训练设置目标向量的。你还会注意到评价表现和计算误差统计量的过程，即 `performance_on_image_list()` 和 `evaluate_performance()`。第一个过程在图片集中迭代，计算其总体平均误差，第二个过程计算针对第一个图片的误差和精度。

你大概率并不会使用到接下来几节中的所有信息，所以不必强迫自己知道代码包干的每一件事情。你只需要将这几节当作代码包的参考指南即可，以便当你需要数据结构和过程的信息时可以查询。

另一个有趣的事情是，如果你还没有尝试本作业的最后一个问题，即使用图片包以图形化的方式来查看连接的权重；你会找到创建并写入图片的过程，如果你想玩一下网络权重的可视化的话。

最后，本作业的目的是让你在神经网络相关的工作中获取到第一手经验；它不是为了让你做 C 程序练习。所以花了很大的努力尽量保持程序简单。如果你需要关于程序是如何工作的澄清，尽管提问。

## 3.3 神经网络包

正如前面提到的，该代码包实现了 3 层全连接前馈神经网络，使用了反向传播权值调整方法。我们以一个对数据结构的简短描述开始，即一个 BPNN (BackPropNeuralNet)。

所有单元的值和权重的值都以 `double` 类型存储在 BPNN。

给定一个 BPNN `*net`，你能用 `net->input_n`、`net->hidden_n` 和 `net->output_n` 分别获取到输入、隐藏以及输出单元的序数。

所有单元都从 1 到  $n$  编号， $n$  是单元在层中的序号。要获取到输入、隐藏及输出层中第  $k$  个单元的值，可以分别使用 `net->input_units[k]`、`net->hidden_units[k]`、或者 `net->output_units[k]`。

这里的目标向量被假定为和输出层的单元一样多，并且可以用 `net->target` 来访问到。第  $k$  个单元的值可以用 `net->target[k]` 来访问到。

要获取到第  $i$  个输入单元连接到第  $j$  个隐藏单元的权重值，可以使用 `net->input_weights[i][j]`。要获取到从第  $j$  个隐藏单元连接到第  $k$  个输出单元的权重值，可以使用 `net->hidden_weights[j][k]`。

程序过程解释如下：

```
void bpn_initialize(seed)
    int seed;
```

这段过程初始化神经网络包。它应该在包中所有其他过程被使用前调用。当前，它存在的唯一目的就是输出 `seed` 来初始化随机数产生器。

```
BPNN *bpnn_create(n_in, n_hidden, n_out)
    int n_in, n_hidden, n_out;
```

使用 `n_in` 个输入单元、`n_hidden` 个隐藏单元、`n_output` 个输出单元来创建一个新的网络。网络中的所有权重都被初始化为在范围  $[-1.0, 1.0]$  中的随机数。它返回一个指向网络结构的指针。如果过程执行失败了，则返回一个 `NULL`。

```
void bpnn_free(net)
    BPNN *net;
```

接收一个指向网络的指针，并将和该网络相关的内存全部清空。

```
void bpnn_train(net, learning_rate, momentum, erro, errh)
    BPNN *net;
    double learning_rate, momentum;
    double *erro, *errh;
```

给定一个指向网络的指针，执行一轮反向传播算法。这里假定输入单元和目标层都已经被恰当地设置过了。`learning_rate` 和 `momentum` 被假定为 0.0 和 1.0 之间的值。`erro` 和 `errh` 是指向双精度类型的指针，分别是输出单元和隐藏单元的  $\delta$  误差值之和。

```
void bpnn_feedforward(net)
    BPNN *net;
```

给定一个指向网络的指针，基于当前的输入值运行该网络。

```
BPNN *bpnn_read(filename)
    char *filename;
```

给定一个文件名，为一个网络分配空间，并使用存储在网络文件中的权重初始化这个网络，然后返回指定这个新的 BPNN 的指针。如果失败就返回 `NULL`。

```
void bpnn_save(net, filename)
    BPNN *net;
    char *filename;
```

给定一个指向网络的指针和一个文件名，将网络保存到该文件中。

### 3.4 图片程序包

图片程序包提供了一组操作 PGM 图片的程序过程。一个图片是由像素组成的矩形栅格；每一个像素是一个 0 到 255 之间的整数。图片使用行与列来进行索引；第 0 行是图片中最上面的一行，第 0 列是图片最左边的一列。

```
IMAGE *img_open(filename)
    char *filename;
```

打开用 `filename` 指定的图片，将其装载到一个新的 `IMAGE` 数据结构中，并返回指向这个新的结构的指针。失败的话就返回 `NULL`。

```
IMAGE *img_creat(filename, nrows, ncols)
    char *filename;
    int nrows, ncols;
```

在内存中创建一个图片，使用给定的文件名，以及 `nrows × ncols` 的维度，并返回一个指向这个图片的指针。所有的像素都被初始化为 0。操作失败就返回 `NULL`。

```
int ROWS(img)
    IMAGE *img;
```

给定指向一个图片的指针，返回这个图片的行数。

```
int COLS(img)
    IMAGE *img;
```

给定指向一个图片的指针，返回这个图片的列数。

```
char *NAME(img)
    IMAGE *img;
```

给定指向一个图片的指针，返回指向它的基本文件名（就是说，如果完整的文件名是 `/usr/joe/stuff/foo.pgm`，就返回指向字符串 `foo.pgm` 的指针）的指针。

```
int img_getpixel(img, row, col)
    IMAGE *img;
    int row, col;
```

给定指向一个图片的指针以及行/列坐标，这个代码过程就返回图片中这些坐标的像素值。

```
void img_setpixel(img, row, col, value)
    IMAGE *img;
    int row, col, value;
```

给定指向一个图片的指针和行/列坐标，以及一个预期在范围 `[0, 255]` 之间的整数值 `value`，这段代码过程就将图片中这个坐标的像素值设置为这个整数值。

```
int img_write(img, filename)
    IMAGE *img;
    char *filename;
```

给定指向一个图片的指针以及文件名，将图片写到磁盘上以该文件名命名的文件中。成功返回 1、失败返回 0。

```
void img_free(img)
    IMAGE *img;
```

给定指向图片的一个指针，撤销其关联的内存分配。

```
IMAGELIST *imgl_alloc()
```

返回指向新的 `IMAGELIST` 结构的指针，其实就是一个指向多个图片的指针数组。给定 `IMAGELIST *il`，`il->n` 就是列表中列表中的图片数量。`il->list[k]` 是指向列表中第 `k` 个图片的指针。

```
void imgl_add(il, img)
    IMAGELIST *il;
    IMAGE *img;
```

给定一个图片列表的指针，以及一个图片指针，就将这个图片添加到图片列表的末尾。



```
void imgl_free(il)
```

```
    IMAGELIST *il;
```

给定指向图片列表的指针，清空它。注意这并不会清空任何该列表所指向的图片。

```
void imgl_load_images_from_textfile(il, filename)
```

```
    IMAGELIST *il;
```

```
    char *filename;
```

接收一个指向图片列表的指针和一个文件名。`filename` 期待是一个包含了一列图片路径的文件，一行一个图片路径。这个列表中的每个图片文件都会被装载到内存中，并添加到图片列表 `il` 里。

### 3.5 hidtopgm

`hidtopgm` 接收如下固定的参数集合：

```
hidtopgm net-file image-file x y n
```

*net-file* 是包含网络的文件，在其中可以找到隐藏单元的权值。

*image-file* 是会被输出的图片文件。

$x$  and  $y$  是用来训练网络的图片的像素维度。

$n$  是目标隐藏单元的数量。 $n$  的范围可以从 1 一直到网络中的隐藏单元数量。

### 3.6 outtopgm

`outtopgm` 接收如下固定的参数集合：

```
outtopgm net-file image-file x y n
```

这和 `hidtopgm` 一样，只不过不是输入单元而是输出单元。确保你指定的  $x$  是 1 加上隐藏单元数，这样你才能获取到与隐藏单元关联的权重以及  $w_0$ 。比如，要查看包含 3 个隐藏单元的网络中的第 2 个输出，就这样做：

```
outtopgm pose.net pose-out2.pgm 4 1 2
```

*net-file* 是包含网络的文件，在其中可以找到隐藏单元的权重。

*image-file* 是会被输出的图片的文件。

$x$  and  $y$  是隐藏单元的维度，其中  $x$  总是 1 + 指定的网络中的隐藏单元数而  $y$  总是 1。

$n$  是目标输出单元数量。 $n$  的范围可以从 1 一直到网络的输出单元数量。